

C# - PHẦN 8.8

GENERIC CONSTRAINTS

Ôn nền tảng - Giải thích nguyên lý - Ví dụ - Bài tập thực hành

GENERIC	CONSTRAINT	.NET
Một khuôn cho nhiều kiểu	Điều kiện dành cho T	Repository, EF Core, DI

Tài liệu học tập có ví dụ C# và bài tập cuối bài

Mục lục nội dung

01. I. Kiến thức nền: Generic Class và vấn đề khi T chưa được giới hạn
02. II. Nhắc lại Interface và đa hình qua interface
03. III. Generic Constraint là gì?
04. IV. Phân tích DisplayService<T>
05. V. Interface Constraint
06. VI. Base Class Constraint
07. VII. So sánh Interface Constraint và Base Class Constraint
08. VIII. where T : class
09. IX. where T : struct
10. X. where T : new()
11. XI. where T : notnull
12. XII. Kết hợp nhiều Constraint
13. XIII. Nguyên lý tổng quát và mức độ cần học
14. XIV. Bảng tóm tắt
15. XV. Bài tập 8.8 - Generic Constraints

Mục tiêu sau khi học xong

Bạn đọc được cú pháp where T : ..., hiểu mỗi constraint cung cấp điều gì cho compiler, biết khi nào cần dùng interface, base class, class, struct, new(), và tự viết được một Generic Class có constraint.

I. Kiến thức nền cần nhớ

1. Generic Class

Generic Class cho phép chúng ta viết một class tổng quát rồi sử dụng với nhiều kiểu dữ liệu khác nhau. T là một kiểu đại diện chưa được xác định tại thời điểm viết class.

```
public class GenericStorage<T>
{
    private readonly List<T> _items = new();

    public void AddItem(T item)
    {
        _items.Add(item);
    }

    public T GetFirstItem()
    {
        return _items[0];
    }
}
```

Khi tạo GenericStorage<int>, T được thay bằng int. Khi tạo GenericStorage<Product>, T được thay bằng Product.

```
GenericStorage<int> numbers = new GenericStorage<int>();
GenericStorage<Product> products = new GenericStorage<Product>();
```

Ý nghĩa cốt lõi

Generic giúp viết một khuôn code dùng lại cho nhiều kiểu, nhưng nếu không có constraint thì compiler gần như chưa biết T có khả năng gì.

2. Vấn đề của Generic không có constraint

Giả sử Generic Class muốn gọi DisplayInfo() trên item:

```
public class DisplayService<T>
{
    public void Display(T item)
    {
        item.DisplayInfo(); // lỗi biên dịch
    }
}
```

Compiler không cho phép vì T có thể là Student, Product nhưng cũng có thể là int, decimal hoặc string. Không phải mọi kiểu đều có DisplayInfo().

```
int number = 10;
// number.DisplayInfo(); // không hợp lệ
```

Generic Constraint sinh ra để cung cấp cho compiler một lời đảm bảo về kiểu T.

II. Nhắc lại Interface

1. Interface là hợp đồng

```
public interface IDisplayable
{
    void DisplayInfo();
}
```

Class nào implement IDisplayable thì bắt buộc phải cung cấp method DisplayInfo().

```
public class Student : IDisplayable
{
    public string Id { get; }
    public string FullName { get; }

    public Student(string id, string fullName)
    {
        Id = id;
        FullName = fullName;
    }

    public void DisplayInfo()
    {
        Console.WriteLine($"Student: {Id} - {FullName}");
    }
}
```

```
public class Product : IDisplayable
{
    public string Id { get; }
    public string Name { get; }

    public Product(string id, string name)
    {
        Id = id;
        Name = name;
    }

    public void DisplayInfo()
    {
        Console.WriteLine($"Product: {Id} - {Name}");
    }
}
```

2. Interface và đa hình

Một biến kiểu interface có thể tham chiếu đến nhiều object thực tế khác nhau, miễn là các class đó implement interface.

```
IDisplayable item1 = new Student("SV01", "Hung");
IDisplayable item2 = new Product("SP01", "Laptop");

item1.DisplayInfo();
item2.DisplayInfo();
```

Liên hệ với constraint

Interface mô tả khả năng chung. Constraint dùng interface để yêu cầu mọi T phải có khả năng đó.

III. Generic Constraint là gì?

Generic Constraint là điều kiện đặt lên kiểu T. Cú pháp tổng quát:

```
where T : điều_kiện
```

Ví dụ:

```
public class DisplayService<T>
    where T : IDisplayable
{
}
```

Đọc là: T phải là một kiểu implement IDisplayable.

Constraint có hai vai trò

- Vai trò 1 - Cánh cổng: quyết định kiểu nào được phép thay thế T.
- Vai trò 2 - Lời cam kết: cho compiler biết mọi T đều có những thành viên được khai báo bởi constraint.

Câu cần nhớ

Constraint không thêm method vào Student hay Product. Nó chỉ quyết định kiểu nào được phép làm T và generic code được phép làm gì với T.

IV. Phân tích DisplayService<T>

```
public class DisplayService<T>
    where T : IDisplayable
{
    public void Display(T item)
    {
        item.DisplayInfo();
    }
}
```

Phân tích từng phần:

- DisplayService<T>: đây là Generic Class.
- where T : IDisplayable: T phải implement IDisplayable.
- Display(T item): method nhận một object kiểu T.
- item.DisplayInfo(): hợp lệ vì compiler biết mọi T đều có DisplayInfo().

Sử dụng với Student

```
DisplayService<Student> studentService =
    new DisplayService<Student>();

Student student = new Student("SV01", "Hung");
studentService.Display(student);
```

Student đáp ứng constraint vì Student implement IDisplayable. Có thể hình dung T được thay bằng Student trong object generic này.

Sử dụng với Product

```
DisplayService<Product> productService =
    new DisplayService<Product>();

Product product = new Product("SP01", "Laptop");
productService.Display(product);
```

Trường hợp không hợp lệ

```
// DisplayService<int> numberService =
//     new DisplayService<int>();
```

int không implement IDisplayable nên bị chặn ngay lúc biên dịch.

Cách diễn đạt chính xác

Student được phép dùng làm kiểu T của DisplayService<T>. Student không thừa hưởng method Display() của DisplayService<T>. Method Display() vẫn thuộc object service.

V. Interface Constraint

Cú pháp:

```
where T : IInterface
```

Ví dụ một interface tính toán:

```

public interface ICalculatable
{
    decimal Calculate();
}

public class CalculationService<T>
    where T : ICalculatable
{
    public decimal GetResult(T item)
    {
        return item.Calculate();
    }
}

```

Interface constraint rất quan trọng vì nó giới hạn theo khả năng. Nhiều class không cùng họ kế thừa vẫn có thể dùng chung generic code nếu cùng implement một interface.

VI. Base Class Constraint

Base class constraint yêu cầu T phải là một class cụ thể hoặc class con của nó.

```

public class Employee
{
    public string FullName { get; }

    public Employee(string fullName)
    {
        FullName = fullName;
    }

    public virtual decimal CalculateSalary()
    {
        return 0;
    }
}

public class OfficeEmployee : Employee
{
    public decimal BaseSalary { get; }

    public OfficeEmployee(string fullName, decimal baseSalary)
        : base(fullName)
    {
        BaseSalary = baseSalary;
    }

    public override decimal CalculateSalary()
    {
        return BaseSalary;
    }
}

public class EmployeeService<T>
    where T : Employee
{
    public void DisplaySalary(T employee)
    {
        Console.WriteLine(employee.FullName);
        Console.WriteLine(employee.CalculateSalary());
    }
}

```

where T : Employee chấp nhận Employee và các class con như OfficeEmployee. Generic code được phép sử dụng các thành viên của Employee.

VII. So sánh Interface Constraint và Base Class Constraint

Loại constraint	Giới hạn theo	Ví dụ kiểu hợp lệ
where T : Employee	Quan hệ kế thừa	Employee, OfficeEmployee, SalesEmployee
where T : IDisplayable	Khả năng/hợp đồng	Student, Product, Employee, Order

Dùng base class khi generic code thật sự cần các dữ liệu hoặc hành vi của một họ kế thừa. Dùng interface khi chỉ cần một khả năng chung và muốn nhiều class không liên quan vẫn dùng được.

VIII. where T : class

```
public class ReferenceStorage<T>
    where T : class
{
    private readonly List<T> _items = new();
}
```

Constraint class yêu cầu T là reference type. Các kiểu như string, Student, Product và Employee phù hợp. int, decimal, bool, DateTime không phù hợp.

Giới hạn quan trọng

where T : class chỉ cho biết T là reference type. Nó không cho biết T có Id, Name hay DisplayInfo(). Muốn dùng các thành viên đó cần interface hoặc base class phù hợp.

IX. where T : struct

```
public class ValueStorage<T>
    where T : struct
{
    public T Value { get; }

    public ValueStorage(T value)
    {
        Value = value;
    }
}
```

Constraint struct yêu cầu T là value type, ví dụ int, decimal, DateTime. Nó không chấp nhận string hoặc các class như Product.

X. where T : new()

Constraint new() yêu cầu T có constructor public không tham số. Nó cho phép generic code tự tạo object bằng new T().

```
public class ObjectFactory<T>
    where T : new()
{
    public T Create()
    {
        return new T();
    }
}
```

Class phù hợp

```
public class Message
{
    public string Content { get; set; }

    public Message()
    {
        Content = "Default message";
    }
}

ObjectFactory<Message> factory =
    new ObjectFactory<Message>();

Message message = factory.Create();
```

Class không phù hợp

```
public class Account
{
    public string Username { get; }

    public Account(string username)
    {
        Username = username;
    }
}

// ObjectFactory<Account> accountFactory =
//     new ObjectFactory<Account>();
```

Account chỉ có constructor có tham số, nên không thể tạo bằng `new Account()` và không đáp ứng `where T : new()`.

Khi nào dùng `new()`?

Chỉ dùng khi generic code thật sự cần tự tạo object T. Nếu generic class chỉ nhận object đã được tạo sẵn qua `Add(T item)`, không cần `new()`.

XI. where T : notnull

```
public class MyDictionary<TKey, TValue>
    where TKey : notnull
{
}
```

Constraint `notnull` thường dùng cho key hoặc các tình huống không muốn type argument là nullable. Ở giai đoạn hiện tại, bạn chỉ cần nhận biết và hiểu ý nghĩa.

XII. Kết hợp nhiều Constraint

```
public interface IEntity
{
    string Id { get; }
}

public interface IDisplayable
{
    void DisplayInfo();
}

public class EntityRepository<T>
    where T : class, IEntity, IDisplayable
{
    private readonly List<T> _items = new();

    public void Add(T item)
    {
        _items.Add(item);
    }

    public void DisplayAll()
    {
        foreach (T item in _items)
        {
            Console.WriteLine(item.Id);
            item.DisplayInfo();
        }
    }
}
```

Phân tích:

- class: T phải là reference type.
- IEntity: compiler cho phép dùng item.Id.
- IDisplayable: compiler cho phép gọi item.DisplayInfo().

Nếu thêm new(), cú pháp phải đặt new() cuối danh sách:

```
where T : class, IEntity, IDisplayable, new()
```

XIII. Nguyên lý tổng quát và mức độ cần học

1. Nguyên lý lựa chọn constraint

Trước khi thêm constraint, hãy hỏi: generic code muốn làm gì với T?

Nhu cầu trong generic code	Constraint phù hợp
Muốn gọi item.DisplayInfo()	where T : IDisplayable
Muốn truy cập item.Id	where T : IEntity
Muốn dùng thành viên Employee	where T : Employee
Muốn gọi new T()	where T : new()
Chỉ cần lưu và trả về T	thường không cần constraint

2. Mức độ cần học cho backend .NET

- Cần hiểu chắc: interface constraint, base class constraint, where T : class.

- Cần hiểu và biết sử dụng: where T : new(), where T : struct, where T : notnull.
- Chưa cần học sâu: unmanaged, Delegate constraint, Generic Math nâng cao.

Trong backend .NET, bạn thường gặp constraint trong Repository, Entity Framework Core, Dependency Injection và các service tổng quát.

XIV. Bảng tóm tắt

Constraint	Điều kiện đối với T	Generic code được phép
where T : class	Reference type	Xử lý T theo reference type
where T : struct	Value type	Chỉ nhận value type
where T : Employee	Employee hoặc class con	Dùng thành viên Employee
where T : IEntity	Implement IEntity	Dùng item.Id
where T : IDisplayable	Implement IDisplayable	Gọi item.DisplayInfo()
where T : new()	Constructor public không tham số	Gọi new T()
where T : notnull	Không nullable	Dùng khi cần khóa/giá trị không-null

Ba câu quan trọng nhất

1) where T : ... là điều kiện dành cho kiểu T. 2) Constraint không thêm method vào entity. 3) Constraint giúp compiler biết generic code được phép làm gì với T.

XV. Bài tập 8.8 - Generic Constraints

Bài tập chia thành hai phần. Phần A luyện interface constraint và class constraint. Phần B luyện new() constraint.

Phần A - DisplayStorage<T>

Bước 1: Tạo interface

```
public interface IDisplayable
{
    void DisplayInfo();
}
```

Bước 2: Tạo class Product

Yêu cầu Product implement IDisplayable và có các property Id, Name, Price.

- Id không được null, rỗng hoặc whitespace.
- Name không được null, rỗng hoặc whitespace.
- Price phải lớn hơn 0.
- DisplayInfo() hiển thị đầy đủ thông tin sản phẩm.

Bước 3: Tạo class Student

Yêu cầu Student implement IDisplayable và có các property Id, FullName, Gpa.

- Id hợp lệ.
- FullName hợp lệ.
- Gpa nằm trong khoảng từ 0 đến 10.
- DisplayInfo() hiển thị đầy đủ thông tin sinh viên.

Bước 4: Tạo Generic Class

```
public class DisplayStorage<T>
    where T : class, IDisplayable
{
    private readonly List<T> _items;

    public DisplayStorage()
    {
        _items = new List<T>();
    }
}
```

Viết các method sau:

- AddItem(T item): kiểm tra null rồi thêm vào danh sách.
- RemoveItem(T item): trả về kết quả bool từ List<T>.Remove().
- GetItemCount(): trả về số phần tử.
- DisplayAll(): nếu rỗng thì thông báo và return; nếu có dữ liệu thì foreach và gọi item.DisplayInfo().

```
public void DisplayAll()
{
    if (_items.Count == 0)
    {
        Console.WriteLine("The storage is empty.");
        return;
    }

    foreach (T item in _items)
    {
        item.DisplayInfo();
    }
}
```

Bước 5: Kiểm tra trong Program

Với Product:

- Tạo DisplayStorage<Product>.
- Thêm ít nhất 3 Product.
- Hiển thị tất cả.
- Xóa một Product.
- Hiển thị lại và in số lượng.

Với Student:

- Tạo DisplayStorage<Student>.
- Thêm ít nhất 3 Student.
- Hiển thị tất cả.
- Xóa một Student.
- Hiển thị lại và in số lượng.

Đặt đoạn sau dưới dạng comment và giải thích vì sao không hợp lệ:

```
// DisplayStorage<int> numbers =
//     new DisplayStorage<int>();
```

Kiểm tra exception null bằng một try-catch riêng:

```
try
{
    products.AddItem(null!);
}
catch (ArgumentNullException ex)
{
    Console.WriteLine(ex.Message);
}
```

Phần B - luyện where T : new()

```
public class ObjectFactory<T>
    where T : new()
{
    public T Create()
    {
        return new T();
    }
}
```

Tạo Message có constructor public không tham số:

```
public class Message
{
    public string Content { get; set; }

    public Message()
    {
        Content = "Default message";
    }

    public override string ToString()
    {
        return Content;
    }
}

ObjectFactory<Message> factory =
    new ObjectFactory<Message>();

Message message = factory.Create();
Console.WriteLine(message);
```

Tạo Account chỉ có constructor có tham số:

```
public class Account
{
    public string Username { get; }

    public Account(string username)
    {
        Username = username;
    }
}
```

Đặt đoạn sau dưới dạng comment và giải thích vì sao không hợp lệ:

```
// ObjectFactory<Account> accountFactory =
//     new ObjectFactory<Account>();
```

Ba câu tự kiểm tra cuối bài

1. Vì sao DisplayStorage<int> không hợp lệ?
2. Vì sao DisplayStorage<Product> được phép gọi item.DisplayInfo()?

3. Vì sao `ObjectFactory<Account>` không hợp lệ nhưng `ObjectFactory<Message>` hợp lệ?

Tiêu chí hoàn thành phần 8.8

Bạn tự giải thích được từng constraint, viết được `DisplayStorage<T>` với interface constraint, viết được `ObjectFactory<T>` với `new()`, và phân biệt constraint với inheritance.

Ghi nhớ nhanh trước khi làm bài

```
where T : IDisplayable
// T phải có DisplayInfo()

where T : IEntity
// T phải có Id

where T : Employee
// T phải là Employee hoặc class con

where T : class
// T phải là reference type

where T : struct
// T phải là value type

where T : new()
// T phải tạo được bằng new T()
```

Một Generic Class không phải lúc nào cũng cần constraint. Chỉ thêm constraint khi logic bên trong cần một bảo đảm cụ thể về T.